INFORME TÉCNICO

Proyecto A.R.A.

Agente de Retornos Autónomo para TechStore

Asignatura	Desarrollo de Software Basado en IA / Ingeniería de Software
Integrantes	Juan Carlos Alvarado Ortega (Arquitecto de Datos y Backend) Constanza Cerda (Ingeniera de Orquestación y Frontend)
Fecha	Noviembre 2026

1. Resumen Ejecutivo

El presente informe documenta el desarrollo inicial del proyecto A.R.A. (Agente de Retornos Autónomo), una solución basada en Inteligencia Artificial diseñada para optimizar y automatizar el proceso de devoluciones de la empresa TechStore. Esta primera fase del desarrollo se centró en la cimentación de una arquitectura de datos robusta, la implementación de un backend funcional y la construcción de la memoria semántica del agente. A través del uso de LangChain, SQLite y ChromaDB, se construyó un entorno capaz de proveer al futuro agente orquestador las herramientas (Tools) necesarias para interactuar con bases de datos, tomar decisiones informadas sobre políticas corporativas y mantener un contexto histórico a largo y corto plazo.

Para consolidar esta arquitectura, la segunda fase del proyecto abordó la orquestación del núcleo del agente y el diseño del frontend. Se implementó un planificador (Planner) configurado con 6 reglas de decisión de negocio y se programó el orquestador principal para conectar las herramientas con los flujos de decisión. Finalmente, se desarrolló una interfaz web interactiva en Streamlit para el usuario final y se garantizó la estabilidad del sistema mediante una suite completa de pruebas automatizadas de punta a punta (E2E).

2. Contexto Organizacional y Requerimiento

TechStore, una cadena retail de productos tecnológicos, enfrenta un desafío operativo significativo: la alta demanda en el procesamiento de devoluciones y garantías. El equipo de servicio al cliente se ve sobrepasado por consultas repetitivas sobre el estado de los pedidos, la validación de políticas y el cálculo de reembolsos.

El requerimiento principal consiste en desarrollar un sistema inteligente que automatice la atención de estos casos, capaz de:

- Identificar y consultar el estado de un pedido en un sistema CRM centralizado.
- Analizar si un producto es elegible para devolución según las políticas de la empresa (plazos de 30 días, penalizaciones, etc.).
- Calcular montos de reembolso y generar etiquetas de envío de forma automatizada.
- Mantener un registro histórico de las interacciones previas con el cliente para ofrecer una experiencia personalizada.
- Acceder de manera autónoma a manuales internos para resolver casos límite.
- Proveer una interfaz web donde los operarios puedan interactuar con el agente y visualizar sus razonamientos frente a casos de prueba obligatorios, como devoluciones estándar, solicitudes fuera de plazo o artículos de alto valor.

3. Arquitectura y Orquestación (Fases 1 a 5)

Para soportar estos requerimientos, se diseñó una arquitectura de backend modular y orientada a herramientas (Tools), preparando el terreno para la orquestación del agente mediante LangChain.

3.1. Stack Tecnológico del Backend

- **Base de Datos Relacional (CRM):** SQLite, utilizado para simular el CRM de TechStore, almacenando datos estructurados como perfiles de clientes, pedidos históricos e historial de atenciones.
- **Base de Datos Vectorial (RAG):** ChromaDB, implementada localmente para almacenar los embeddings del manual de políticas de devoluciones.
- **Modelos de Embeddings:** sentence-transformers (modelo all-MiniLM-L6-v2) operado a través de HuggingFaceEmbeddings de LangChain. Se optó por una solución 100% local para evitar dependencias de APIs externas y asegurar la privacidad de los documentos corporativos.
- **Testing:** pytest, garantizando la integridad de cada módulo mediante pruebas unitarias.

3.2. Diagrama de Arquitectura

El siguiente diagrama detalla la arquitectura de las herramientas desarrolladas por el equipo de Backend:

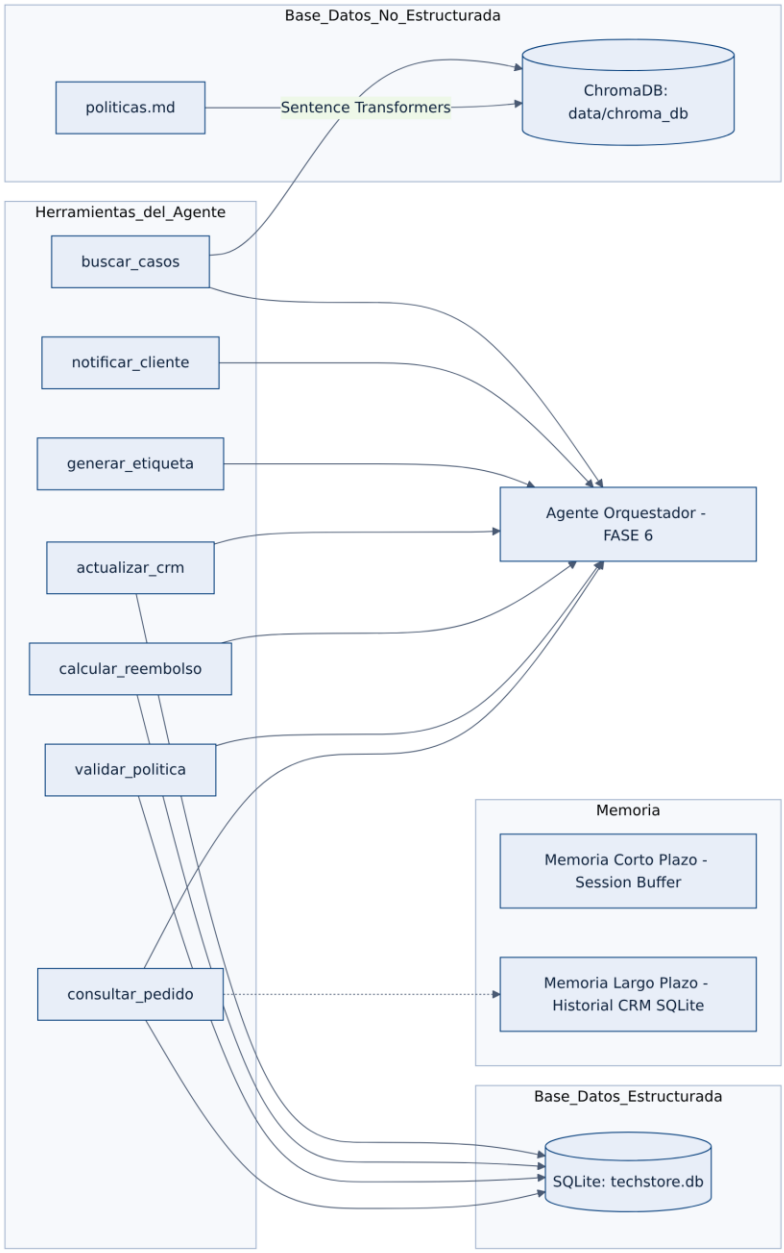


Figura 1. Arquitectura de las herramientas del backend

3.3. Diagrama de Orquestación General

Diagrama de orquestación del agente:

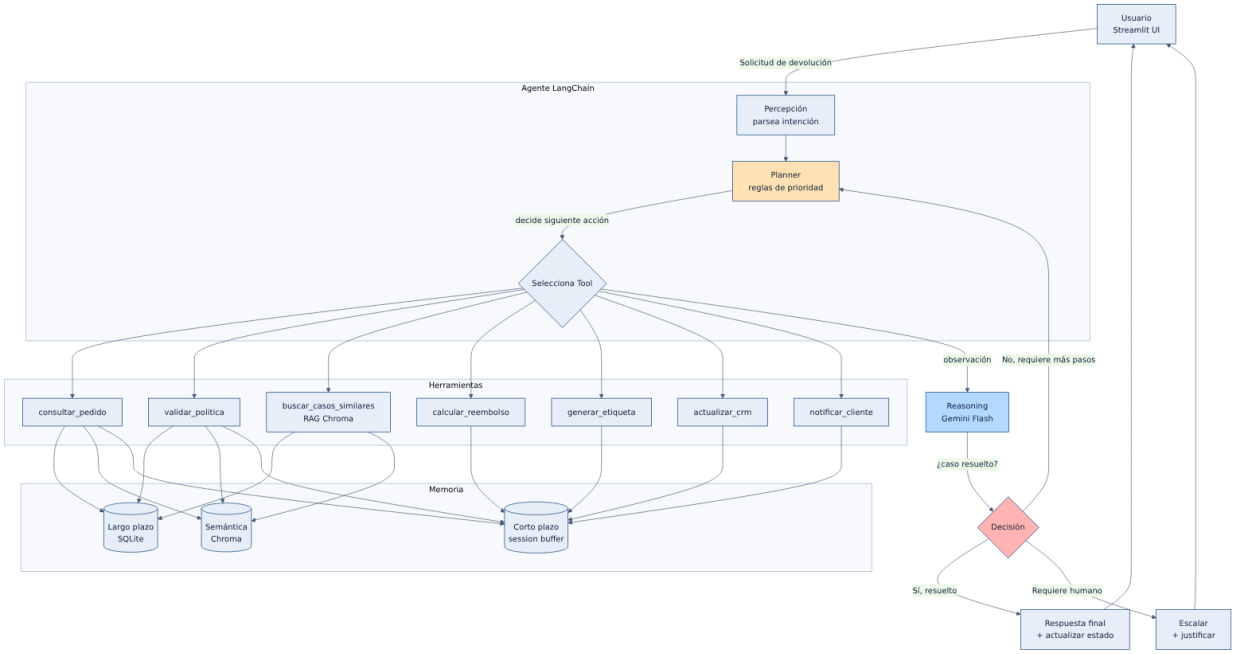


Figura 2. Orquestación general del agente

4. Memoria y Recuperación de Contexto

El sistema dota al agente de dos niveles de memoria:

- **Corto Plazo (`short_term.py`):** Implementada mediante ConversationBufferMemory de LangChain. Conserva el contexto de la interacción en curso, permitiendo que la futura interfaz en Streamlit mantenga un flujo conversacional coherente.
- **Largo Plazo (`long_term.py`):** Permite al agente recuperar interacciones previas del cliente. Esta funcionalidad se inyectó directamente en la herramienta `consultar_pedido`, de modo que al buscar un ID de pedido, el agente no solo obtiene los datos transaccionales, sino también todo el historial de soporte almacenado en el CRM (SQLite), permitiendo decisiones fundamentadas en el comportamiento histórico del usuario.

5. Planificación y Toma de Decisiones (Tools)

El núcleo de la lógica de negocio se encapsuló en 7 herramientas específicas que el agente podrá invocar autónomamente:

- **`consultar_pedido`**: Cruza datos transaccionales con el historial del cliente.
- **`validar_politica`**: Aplica reglas duras calculando plazos de vencimiento desde la fecha de compra.

- `calcular_reembolso`: Aplica penalizaciones porcentuales según el tipo de devolución.
- `generar_etiqueta`: Emula la creación de códigos UUID para retornos físicos.
- `actualizar_crm`: Actualiza el estado del ticket directamente en SQLite.
- `notificar_cliente`: Registra interacciones de salida.
- `buscar_casos_similares`: (RAG) Herramienta semántica que busca en la base vectorial directrices para casos no contemplados en las reglas duras, permitiendo al agente razonar sobre políticas corporativas textuales.

5.1. Planificación y Orquestación UI

El backend fue conectado al usuario mediante una arquitectura de orquestación en `agent/core.py` y un `planner.py`.

El agente utiliza un motor de razonamiento para evaluar el contexto y aplicar 6 reglas de negocio estrictas (ej. escalar si supera los 30 días, solicitar aprobación para montos altos). Todo este flujo se visualiza a través de una interfaz gráfica construida con Streamlit (`app.py`), la cual permite al usuario ingresar el ID del pedido y revisar en tiempo real el log de razonamiento (trace) del agente.

6. Resultados y Evidencias

Durante el desarrollo, se logró completar exitosamente la arquitectura del backend. El hito principal fue la superación del entorno de pruebas unitarias (pytest), confirmando el funcionamiento robusto de todas las herramientas.

- Se implementó exitosamente un pipeline de generación de datos mockeados para simular los pedidos.
- Se resolvieron conflictos críticos de dependencias con la API de Google, optando por una migración hacia modelos de embeddings locales (sentence-transformers), logrando una base vectorial funcional y desvinculada de servicios en la nube de pago.
- **Evidencia Técnica:** La suite de pruebas reportó 7 tests en verde (7 passed), validando desde la consulta básica a SQLite hasta la búsqueda semántica en ChromaDB. (Se adjuntan capturas de pantalla de la terminal en los anexos del repositorio).

6.1. Evidencias de Interfaz y Pruebas E2E

Se completó exitosamente la orquestación y la interfaz de usuario.

- **Pruebas End-to-End (pytest):** Se ejecutó la suite completa de pruebas (12+ tests en verde), validando el flujo de punta a punta y utilizando un fixture (`conftest.py`) para limpiar la base de datos entre ejecuciones.

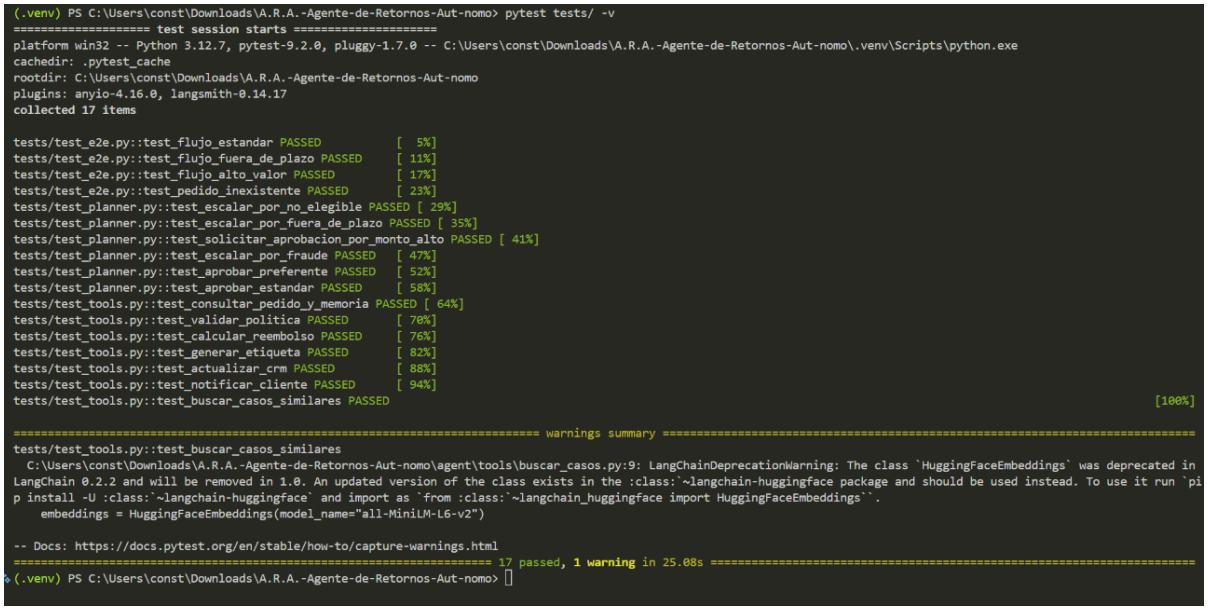


Figura 3. Ejecución de la suite de pruebas (pytest)

- **Validación de Interfaz:** Se validaron los 3 escenarios exigidos en la plataforma Streamlit. *(Caso estándar PED-001, Fuera de plazo PED-015 y Alto valor PED-022)*.

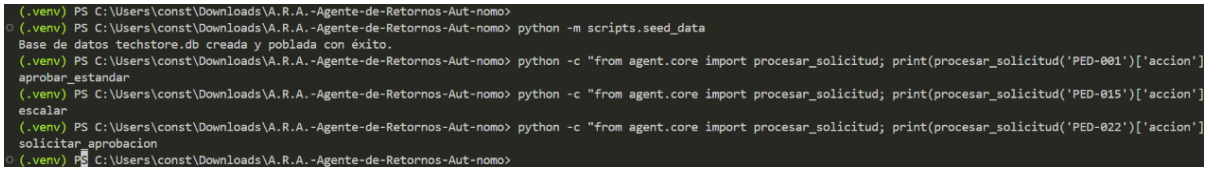


Figura 4. Resultado de los tres escenarios de prueba

7. Conclusiones y Reflexiones Individuales

Reflexión de Juan Carlos Alvarado

Viniendo del área de urgencias, donde las decisiones hay que tomarlas rápido y con la cabeza fría, hacer este backend me exigió lo mismo pero frente al computador. Estar como dos horas pegado con los errores de código y los problemas con las librerías fue difícil, mucha tensión como cuando llega un paciente grave y los equipos fallan. Y la verdad, al principio me equivoqué en insistir en usar la inteligencia de Google para guardar los datos, lo que resultó en una gran pérdida de tiempo.

Pero en salud uno aprende que desesperarse no sirve de nada: hay que frenar, mirar qué está fallando y cambiar el rumbo. Ahí decidí dejar de complicarme con cosas externas y armar todo local con los transformers, que fue la mejor decisión para salvar el proyecto. Ver que al final los 7 tests pasaron la prueba con color verde y así de este modo terminar la Fase 5, después de tanto pelear con la pantalla fue un alivio . Al final en esta área de informática ,el tener la posibilidad de poder armar este tipo de sistemas me gusta mucho en lo personal; ver que la

lógica funciona y que todo encaja bien es lo que hace que valga la pena cada hora frente al notebook.

Reflexión de Constanza Cerda

Como Administradora de Empresas de formación, siempre he evaluado los procesos desde una perspectiva de eficiencia y control financiero, pero programar la orquestación de este agente llevó ese desafío a un nivel técnico completamente nuevo. Tomar las siete herramientas del backend que construyó Juan Carlos y lograr que el "cerebro" del sistema (el Planner) tomara las decisiones correctas bajo reglas estrictas fue un verdadero rompecabezas. Pasé horas tratando de entender por qué las pruebas automatizadas fallaban al ejecutarse dos veces seguidas, hasta que logré dominar la inyección del fixture de limpieza en conftest.py para reiniciar la base de datos. Hubo momentos de frustración, especialmente peleando con el manejo del estado en Streamlit para que la interfaz web mostrara el razonamiento del agente de forma transparente y fluida. Sin embargo, estudiar ingeniería en formato vespertino te enseña a tener resiliencia y a buscar soluciones analíticas frente a cada bloqueo. Cuando finalmente vi la aplicación funcionando en mi navegador local, aprobando el caso estándar y escalando correctamente las devoluciones fuera de plazo y de alto valor, sentí una inmensa satisfacción. Lograr que el código abstracto se convierta en una solución visual e inteligente hace que cada madrugada frente a la pantalla valga totalmente la pena.

8. Referencias APA y Declaración de Uso de IA

Referencias APA

- LangChain. (2024). *Agents and Tool Calling*. Recuperado de https://python.langchain.com/docs/modules/agents/
- Streamlit. (2024). *Streamlit Documentation*. Recuperado de https://docs.streamlit.io/
- Google. (2024). *Gemini API Documentation*. Recuperado de https://ai.google.dev/docs

Declaración de uso de IA

Se utilizó asistencia de Modelos de Lenguaje Grandes (LLMs) durante el proceso de codificación para la generación de scripts de prueba (pytest), resolución de conflictos de dependencias (downgrades y debugging de librerías de LangChain), y sugerencias de estructura de código para la integración de bases vectoriales. Ningún texto de la sección de reflexiones personales fue generado por IA.

Herramienta	Uso
Gemini Flash	Motor de razonamiento principal del agente y toma de decisiones en `core.py`.

Herramienta	Uso
LLMs locales (Sentence-transformers)	Base vectorial para ChromaDB y RAG (Fase 5).
Gemini	Apoyo en la sintaxis de tests End-to-End (`pytest`), estructuración de UI en Streamlit y maquetación del diagrama Mermaid.